Deep Neural Network (RNN - Recurrent Neural Network)

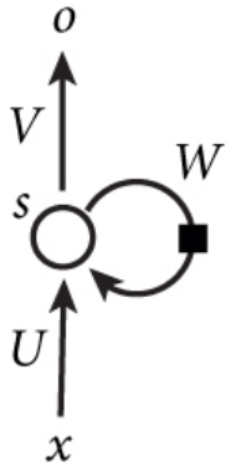
Recurrent Neural Network Structure

Recurrent Neural Network Functions

Recurrent Neural Network Training

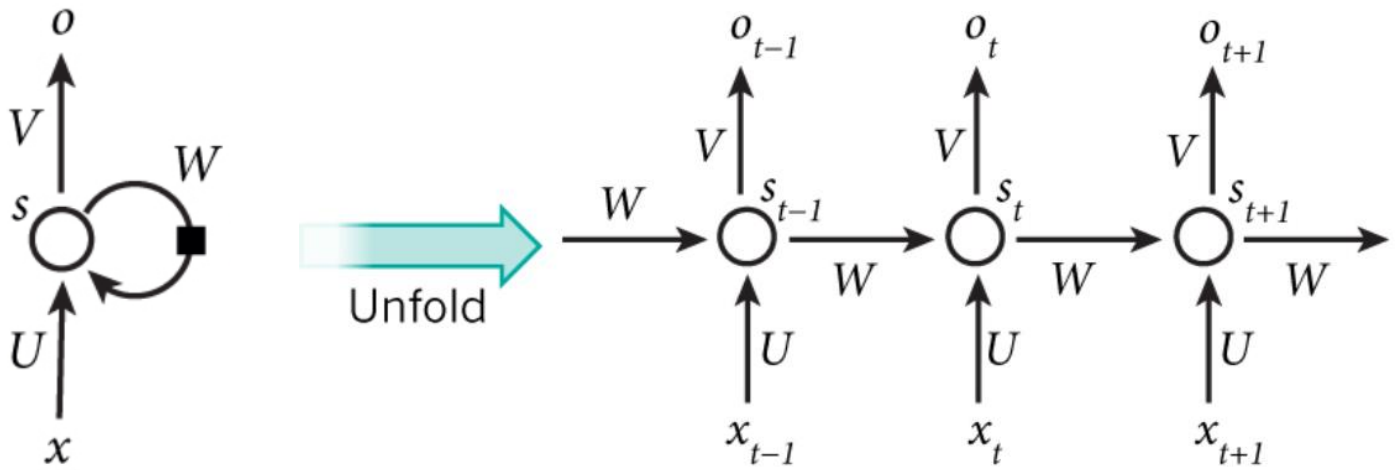
Contents

- RNN Introduction
- BPTT for RNN Training
- LSTM/GRU Model



RNN – Architecture

- **Vanilla RNN**

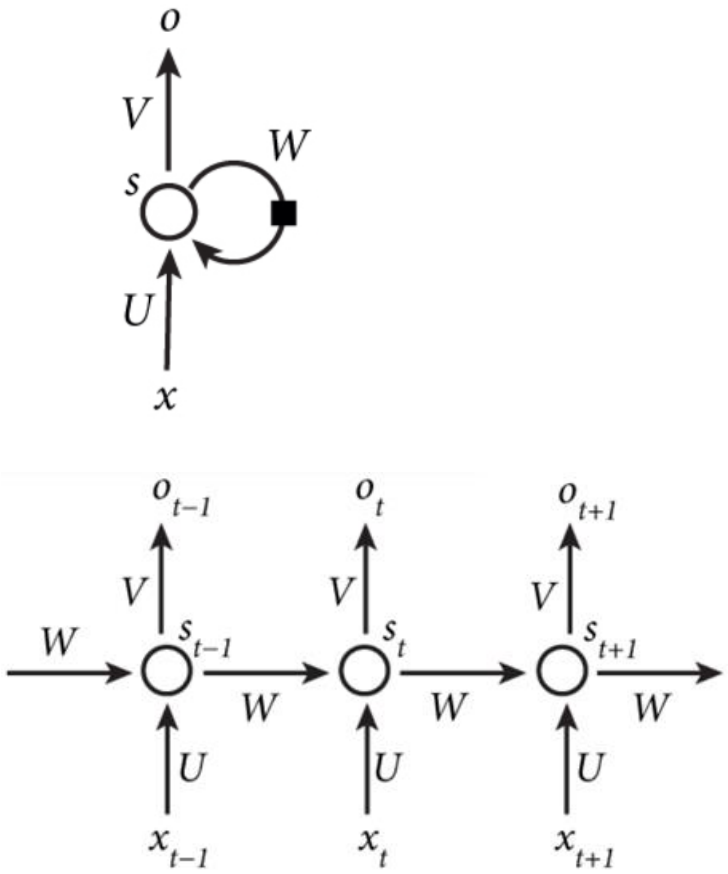


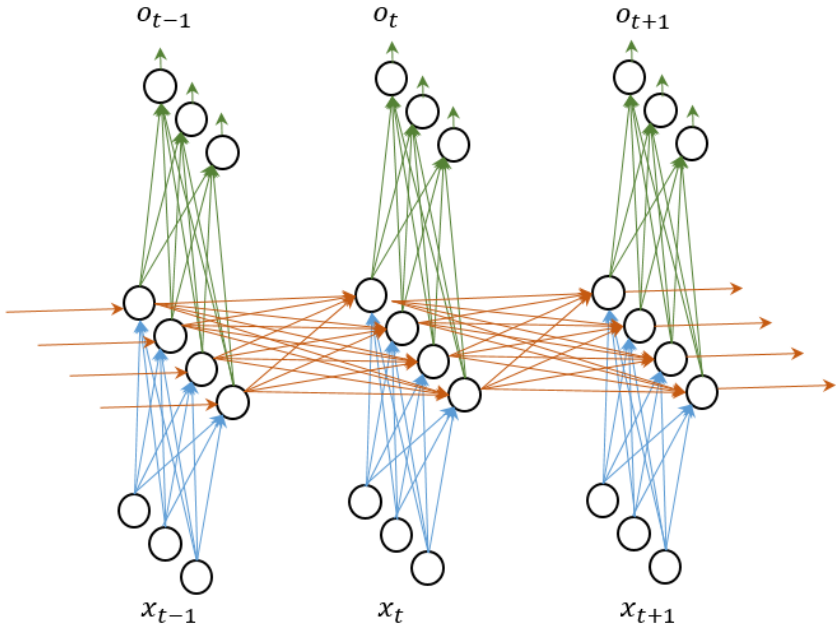
- **input sequence:** $x_1, x_2, \dots, x_t, \dots, x_T$
- **output sequence:** $o_1, o_2, \dots, o_t, \dots, o_T$
- **states:** $s_0, s_1, s_2, \dots, s_t, \dots, s_T$
- **weights:** U, W, V

$$s_t = \tanh(Ux_t + Ws_{t-1})$$

$$o_t = \text{softmax}(Vs_t)$$

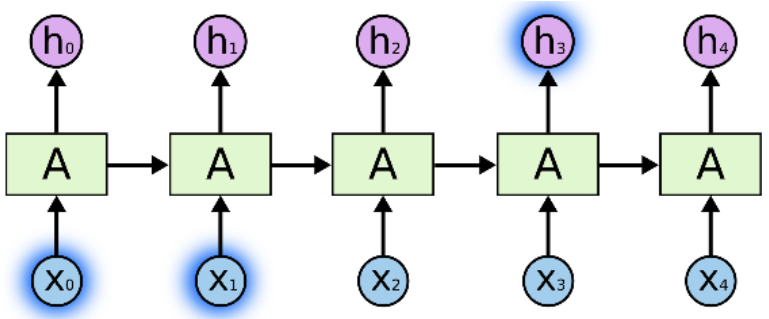
RNN – Architecture



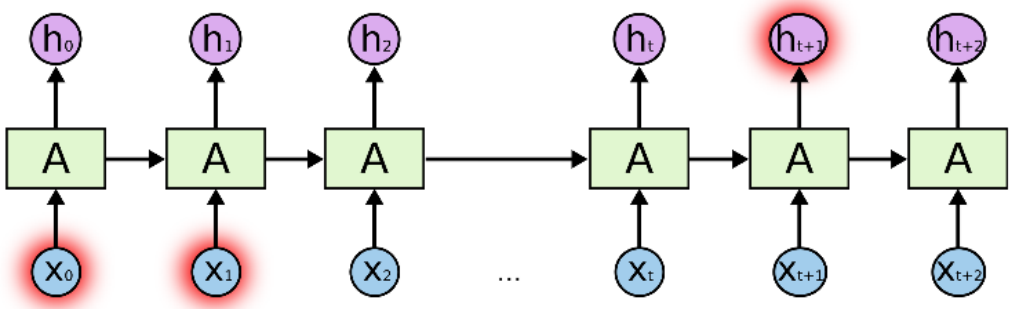


Useful for Temporal Dependencies

- **Short-term dependency**

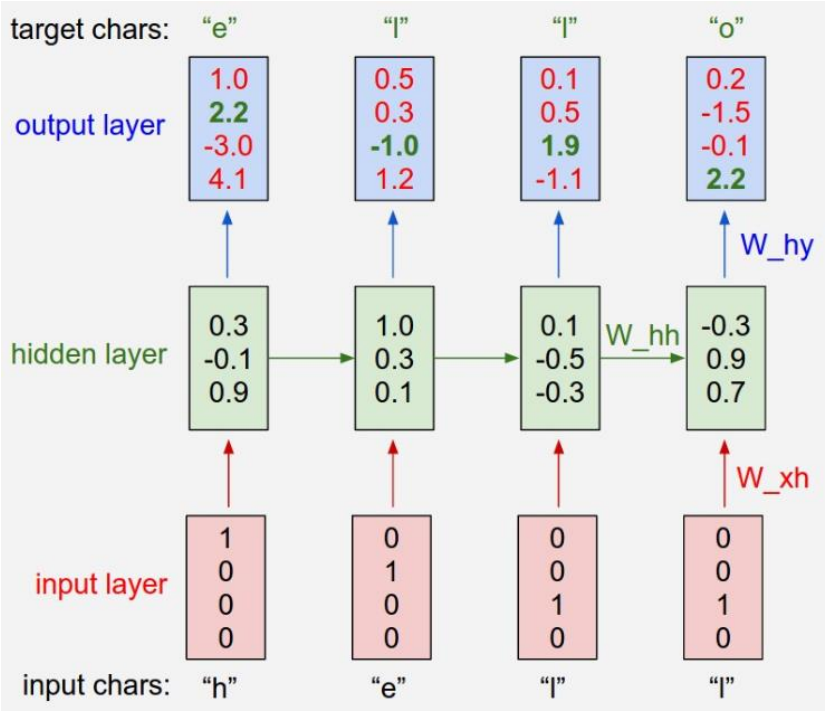


- **Long-term dependency**



RNN Example

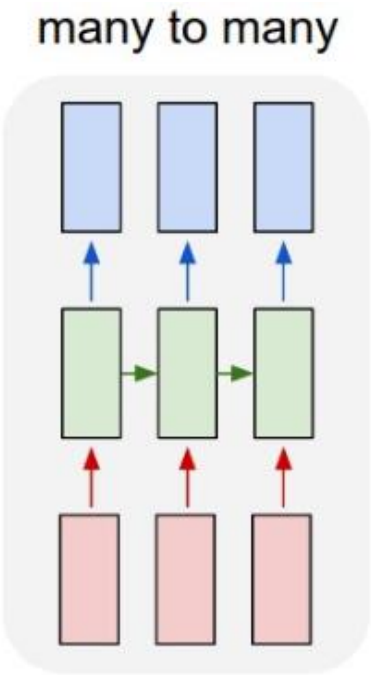
- **Character-level language model**



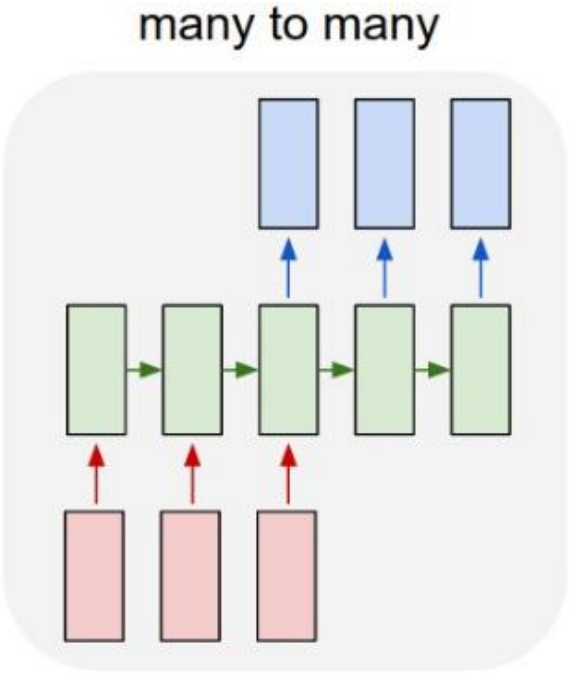
An example RNN with 4-dimensional input and output layers, and a hidden layer of 3 units (neurons). This diagram shows the activations in the forward pass when the RNN is fed the characters "hell" as input. The output layer contains confidences the RNN assigns for the next character (vocabulary is "h,e,l,o"); We want the green numbers to be high and red numbers to be low.

Flexible RNN Architectures (1)

- **many-to-many**



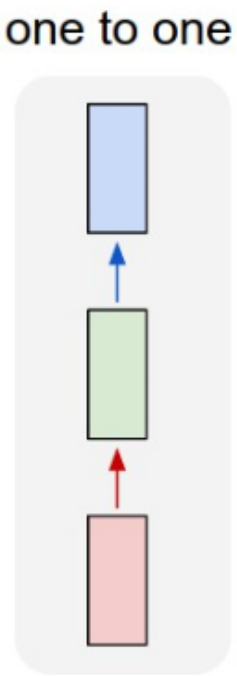
e.g. Video Classification
Image Frames to Classes



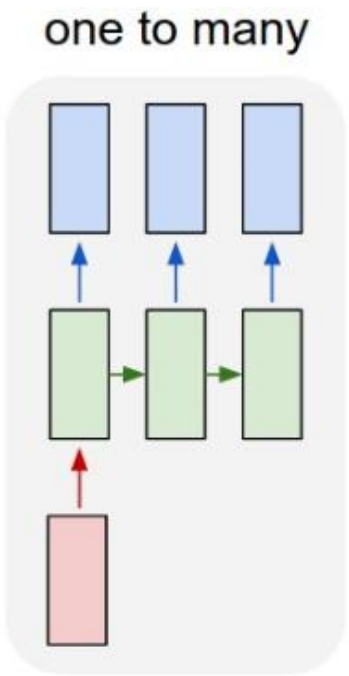
e.g. Machine Translation
Seq. of Words to Seq. of Words

Flexible RNN Architectures (2)

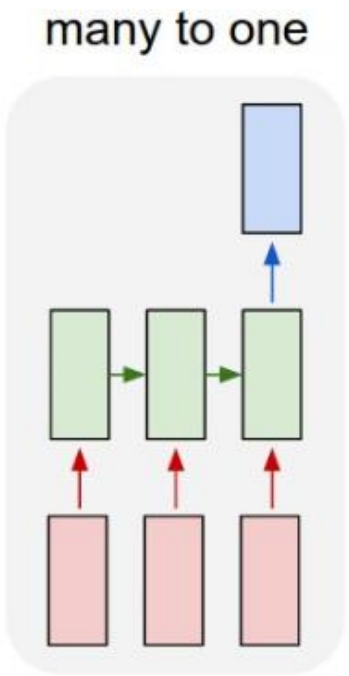
- **one-to-many or many-to-one**



e.g. Image Classification
without RNN



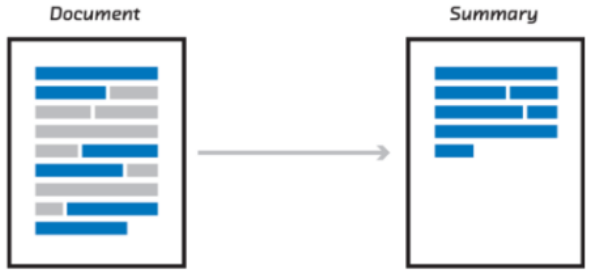
e.g. Image Captioning
Image to Sequence of Words

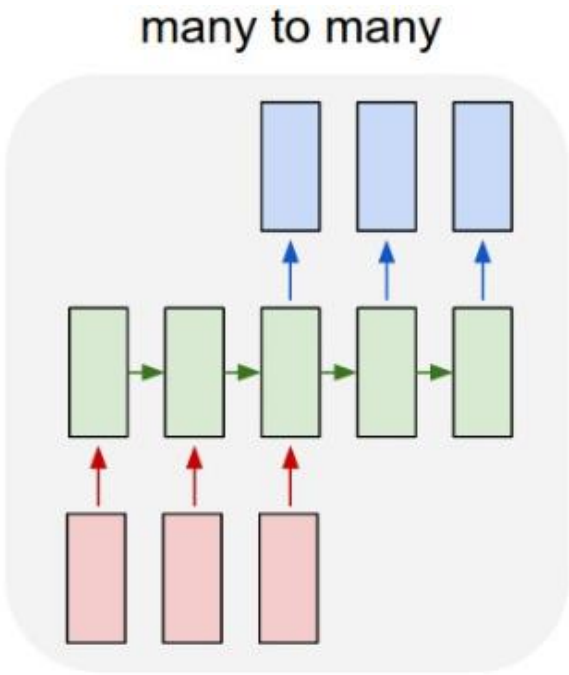


e.g. Sentiment Classification
Sequence of Words to Sentiment

RNN - Applications

- **many-to-many**
  - **Text Summarization**





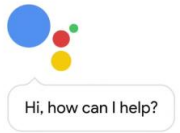
  - **Machine Translation**



  - **Speech Recognition**











RNN - Applications

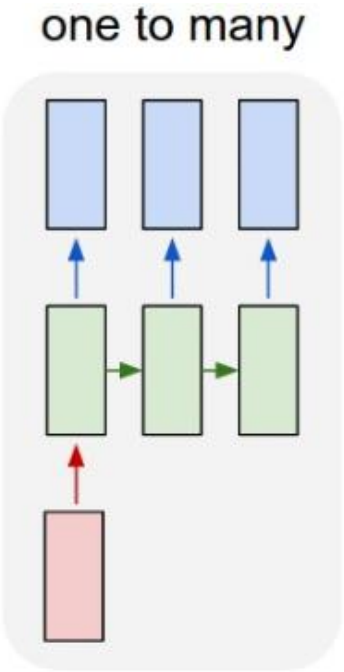
- **one-to-many**
  - **Image Captioning**



"black and white dog jumps over bar."



"little girl is eating piece of cake."



e.g. Image Captioning
Image to Sequence of Words

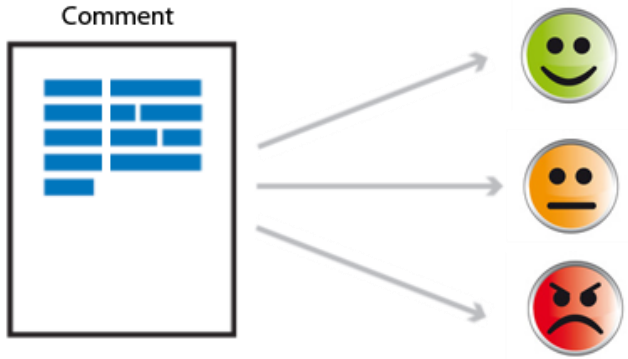
[1] Andrej Karpathy, Li Fei-Fei, “Deep Visual-Semantic Alignments for Generating Image Descriptions”, CVPR (2015)

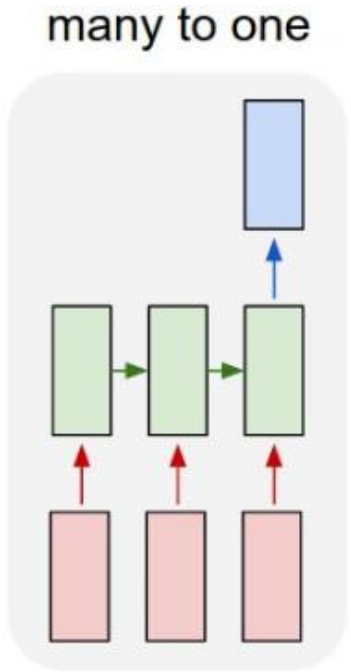
RNN - Applications

- **many-to-one**
  - **Motion in Video**



  - **Sequence of Words to Sentiment**



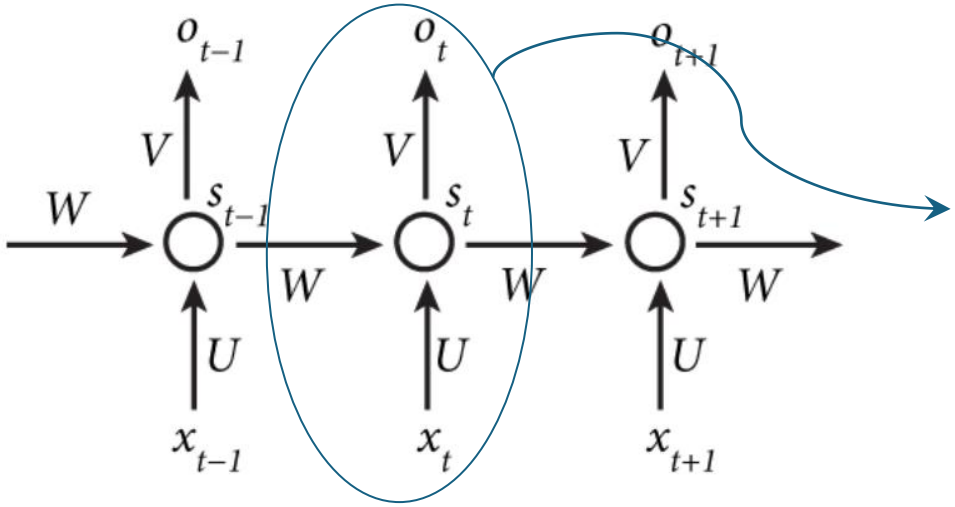


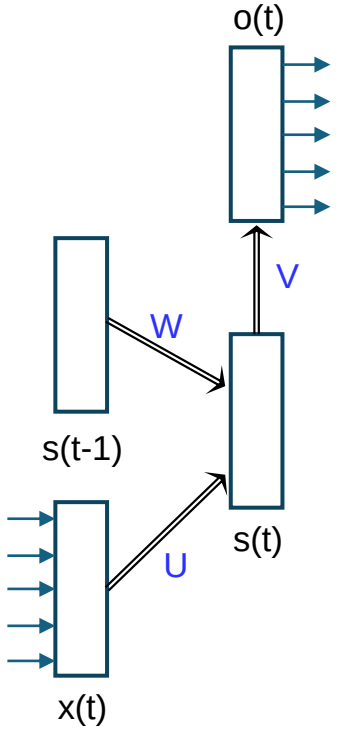
e.g. Sentiment Classification…
Sequence of Words to Sentiment

BPTT for RNN training

Back-Propagation Through Time

- **Based on Standard BP**



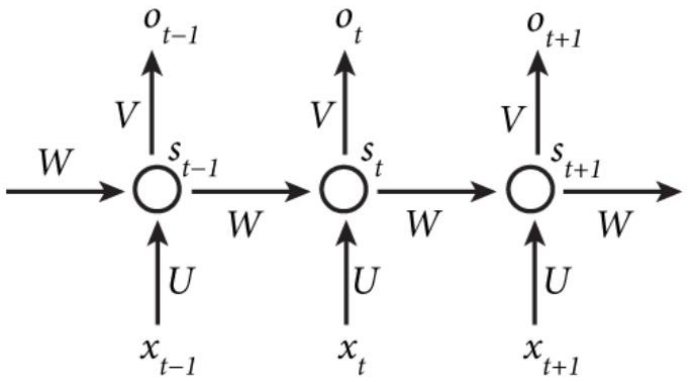


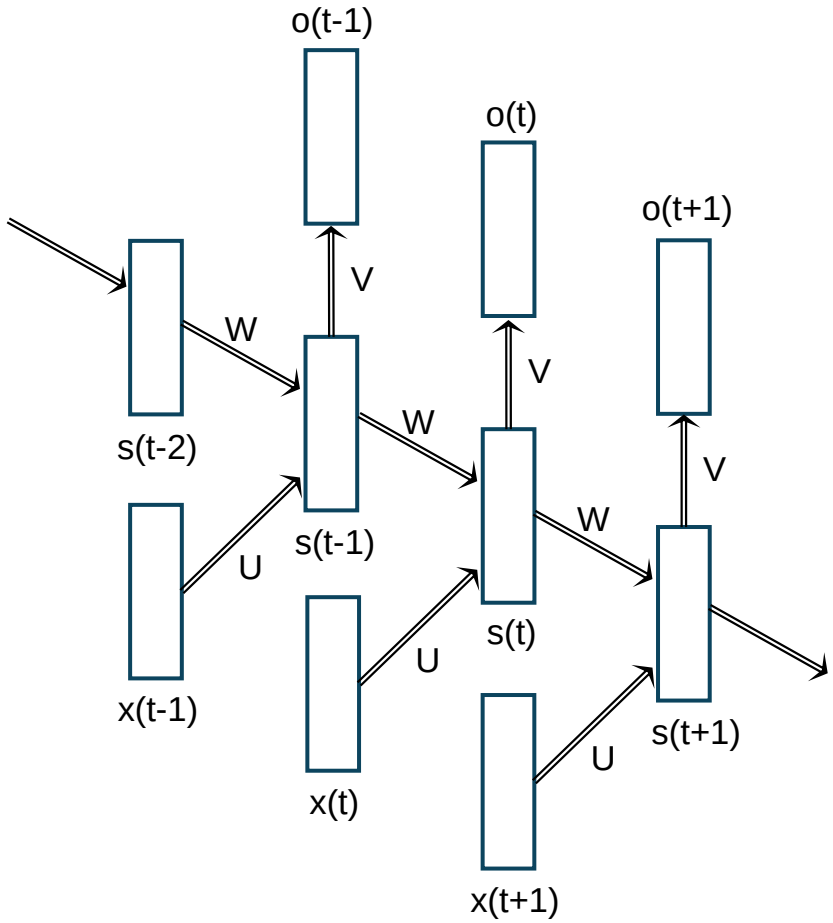
$$s_t = \tanh(Ux_t + Ws_{t-1})$$

$$o_t = \mathrm{softmax}(Vs_t)$$

Back-Propagation Through Time

- **Unfolded RNN is a Feed-forward Network**





Back-Propagation Through Time

- **Training sample is a time series data**

$$\mathbf{X} = (< \mathrm{x}(1), \mathrm{x}(2), \ldots, \mathrm{x}(T) >, < \mathrm{d}(1), \mathrm{d}(2), \ldots, \mathrm{d}(T) >)$$

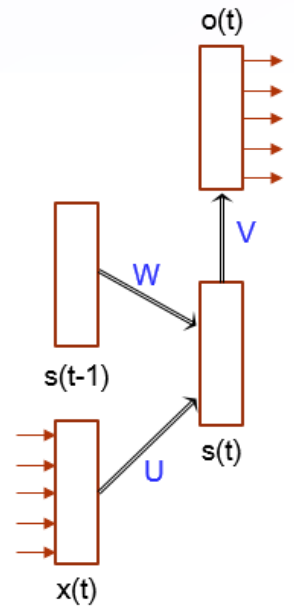
where $\mathrm{x}(t) \in R^K$, $\mathrm{d}(t) \in R^L$

- **Forward calculations, and Errors**

$$\mathrm{s}(t) = \tanh\big(\mathrm{U} \cdot \mathrm{x}(t) + \mathrm{W} \cdot \mathrm{s}(t-1)\big), \text{where } \mathrm{s}(0) = \mathbf{0} \in R^N$$

$$\mathrm{o}(t) = \mathrm{softmax}\big(\mathrm{V} \cdot \mathrm{s}(t)\big)$$

$$E(t) = \frac{1}{2}\|\mathrm{o}(t) - \mathrm{d}(t)\|^2, \qquad t = 1, 2, \ldots, T$$



Back-Propagation Through Time

- **Error back-propagation**

  - j^{th} **output node at** T**:**

  - i^{th} **hidden node at** T**:**

  - j^{th} **output node at** $t(\neq T)$**:**

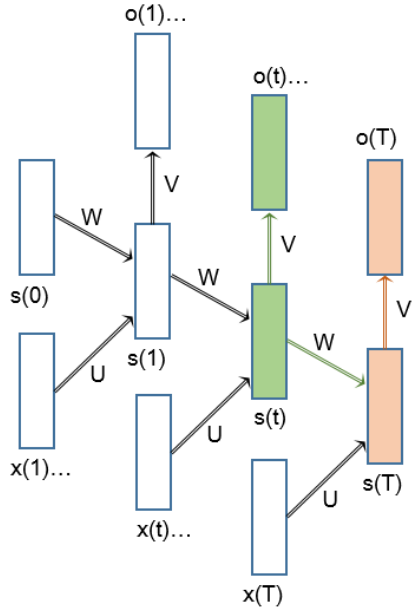
  - i^{th} **hidden node at** $t(\neq T)$**:**

$$\delta_j(T) = f_1'\left(net_j(T)\right)\left(o_j(T) - d_j(T)\right)$$

$$\delta_i(T) = f_2'\left(net_i(T)\right)\left(\sum_{j=1}^{L}\delta_j(T)v_{ij}\right)$$

$$\delta_j(t) = f_1'\left(net_j(t)\right)\left(o_j(t) - d_j(t)\right)$$

$$\delta_i(t) = f_2'\left(net_i(t)\right)\left(\sum_{j=1}^{N}\delta_j(t+1)w_{ij} + \sum_{j=1}^{L}\delta_j(t)v_{ij}\right)$$



Back-Propagation Through Time

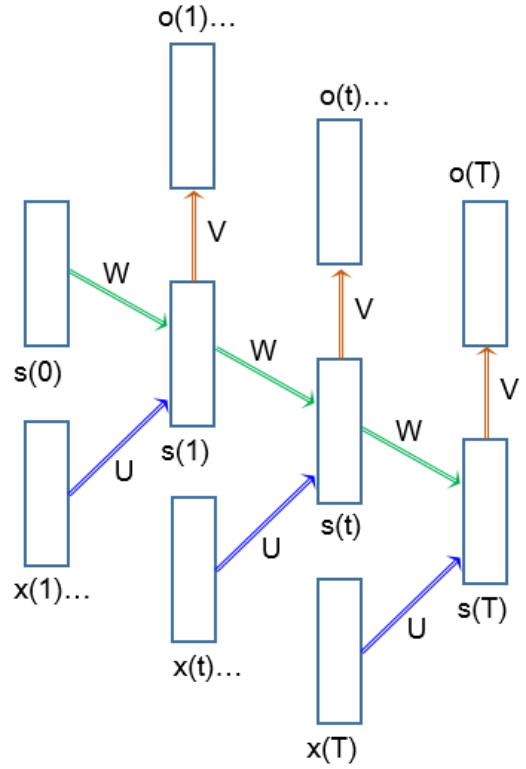
- **Weight update w.r.t. the given training sample**

X =($< x(1), x(2), \ldots, x(T) >$, $< d(1), d(2), \ldots, d(T) >$)

$$v_{ij} = v_{ij} - \rho \frac{\partial E}{\partial v_{ij}} = v_{ij} - \rho \sum_{t=1}^{T} \delta_i(t)\, s_i(t)$$

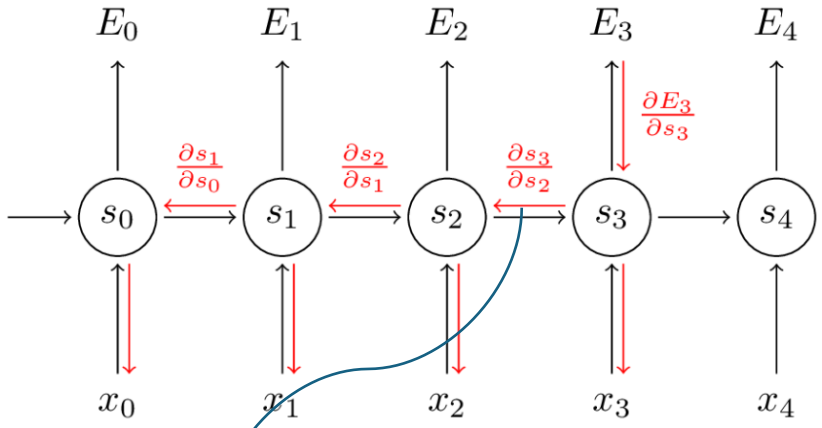
$$w_{ij} = w_{ij} - \rho \frac{\partial E}{\partial w_{ij}} = w_{ij} - \rho \sum_{t=1}^{T} \delta_i(t)\, s_i(t-1)$$

$$u_{ij} = u_{ij} - \rho \frac{\partial E}{\partial u_{ij}} = u_{ij} - \rho \sum_{t=1}^{T} \delta_i(t)\, x_i(t)$$



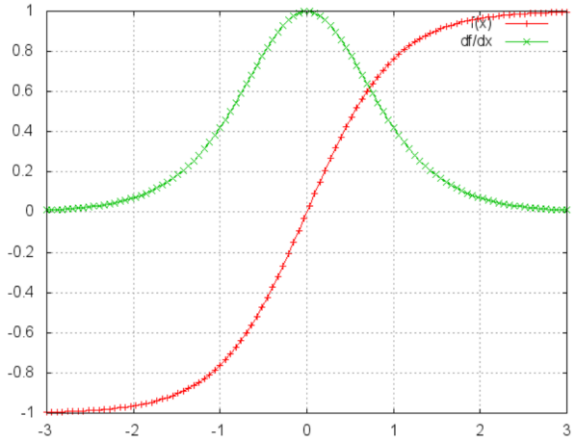
Vanishing Gradient Problem

- **Gradients saturate to 0 as back-propagation proceeds**



$$\delta_i(t) = f_2'\big(net_i(t)\big)\left(\sum_{j=1}^{N}\delta_j(t+1)w_{ij} + \sum_{j=1}^{L}\delta_j(t)v_{ij}\right)$$

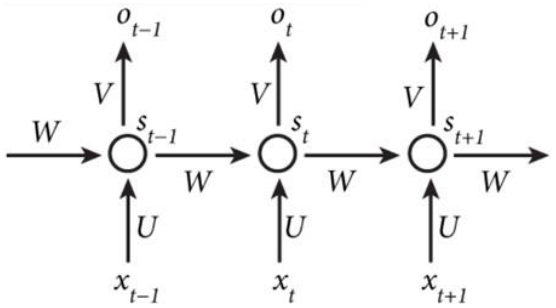
- **tanh and its derivatives**

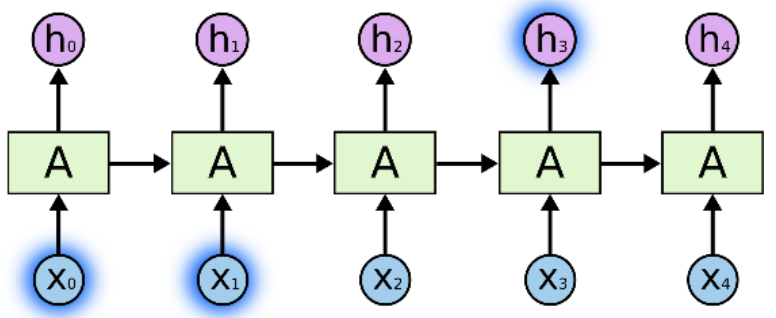


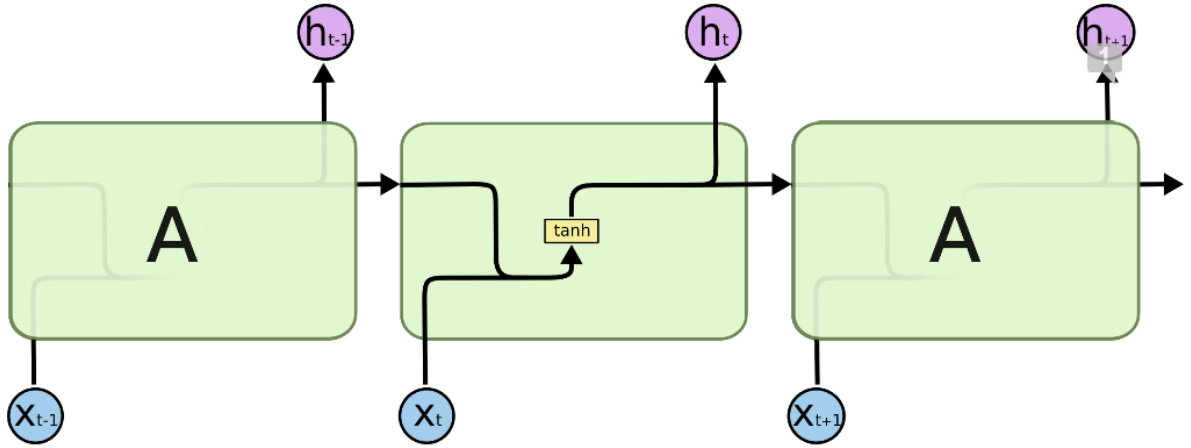
LSTM Network

Vanilla RNN

- A chain of repeating cells

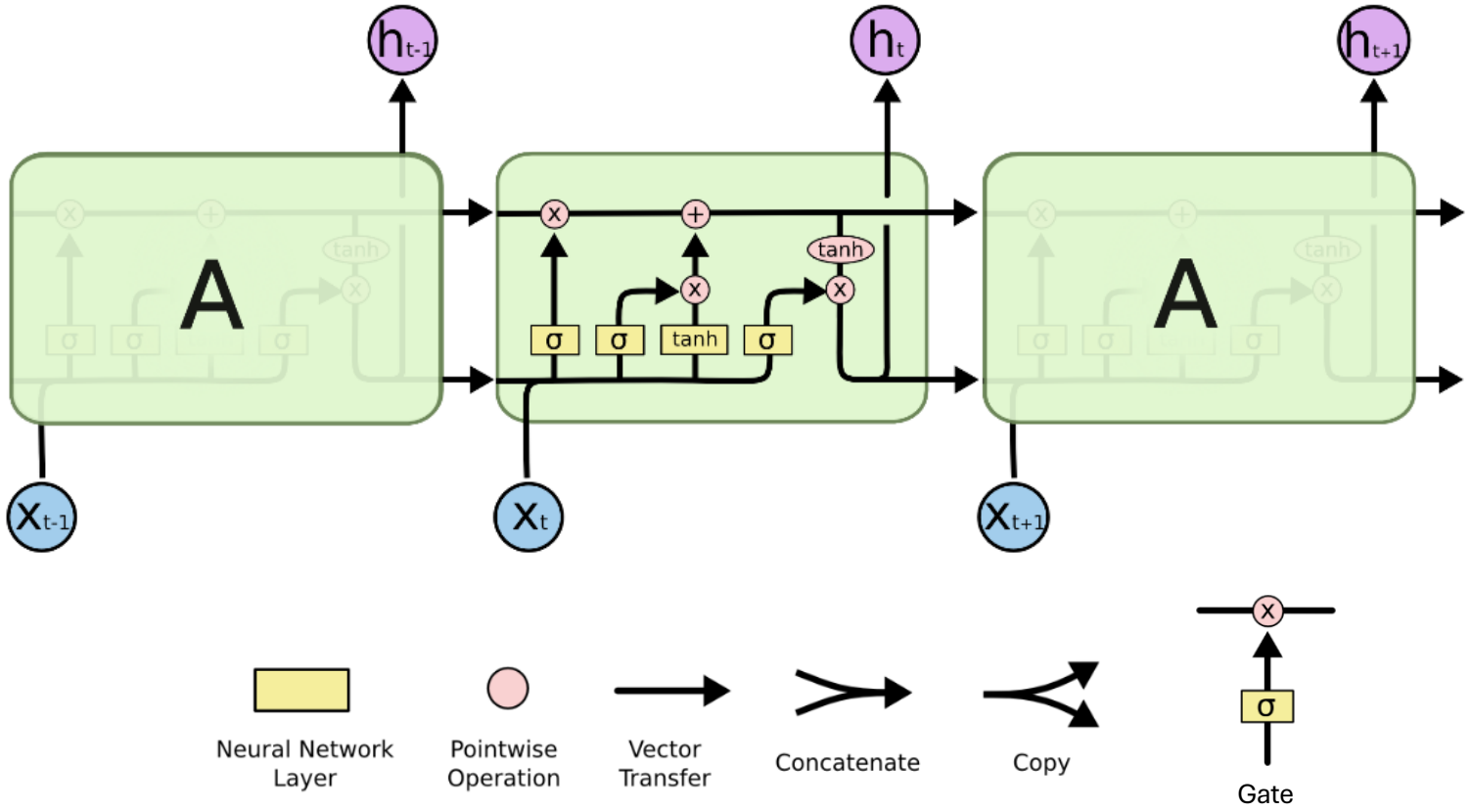






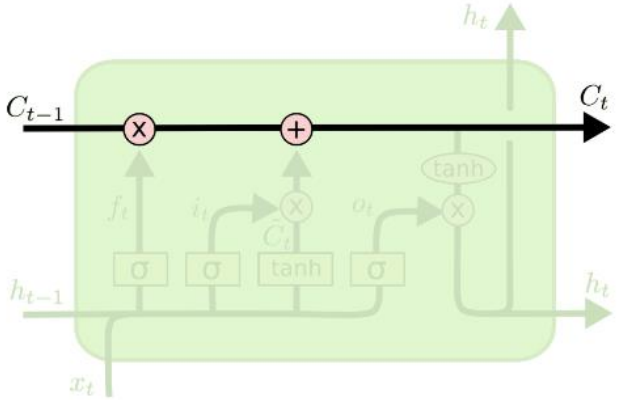
LSTM for Long-Term Dependencies

- **Hochreiter & Schmidhur (1997)**

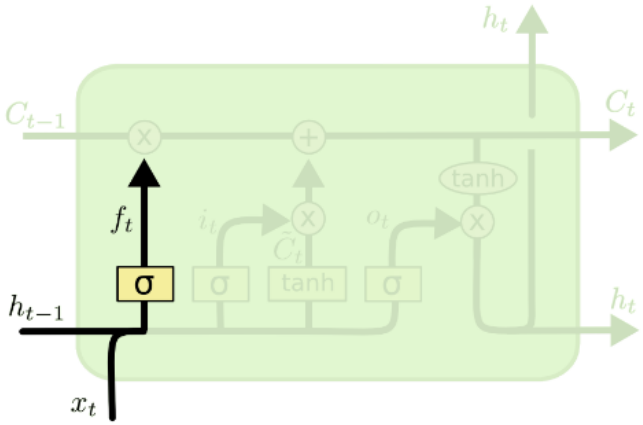


LSTM Cell (1)

- **Cell states**



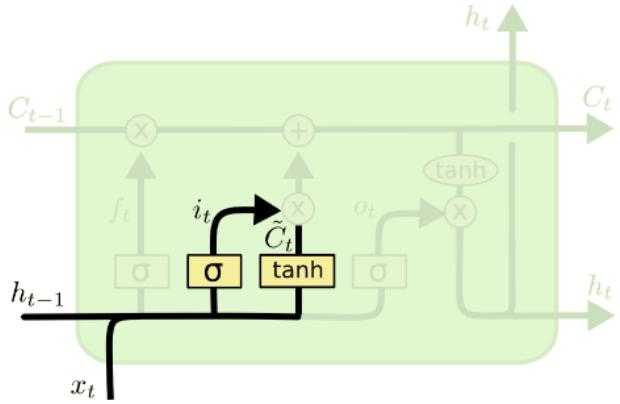
- **'Forget' gate layer**



$$f_t = \sigma\left(W_f \cdot [h_{t-1}, x_t] \; + \; b_f\right)$$

LSTM Cell (2)

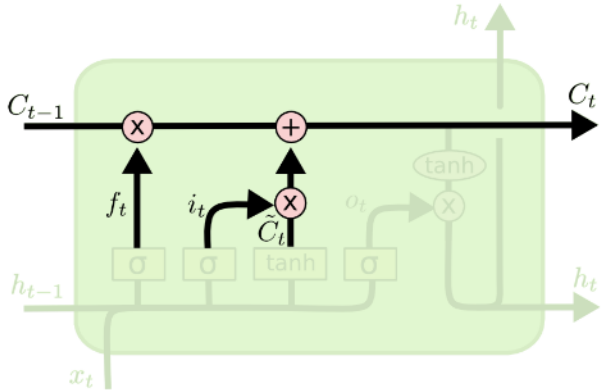
- ‘Input’ gate layer



$$i_t = \sigma\left(W_i \cdot [h_{t-1}, x_t] \; + \; b_i\right)$$

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] \; + \; b_C)$$

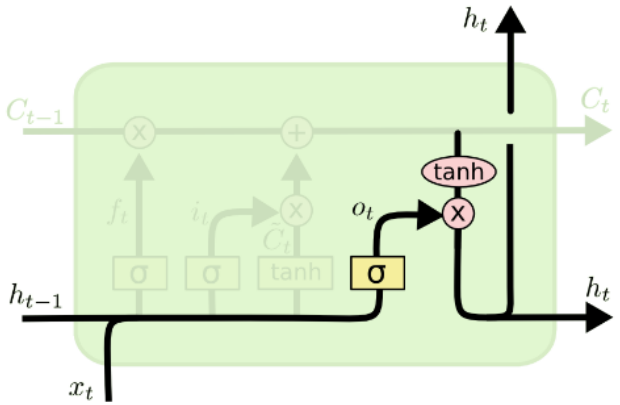
- Update the cell state



$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

LSTM Cell (3)

- ‘Output’ gate layer

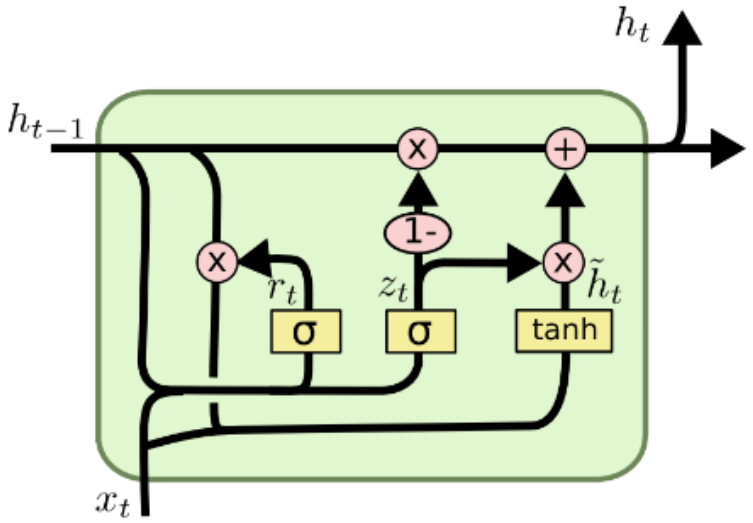


$$o_t = \sigma\left(W_o\left[h_{t-1}, x_t\right] + b_o\right)$$

$$h_t = o_t * \tanh\left(C_t\right)$$

GRU Network

- **A Variant of LSTM Network [Cho & Bengio, 2014]**



$$z_t = \sigma\left(W_z \cdot [h_{t-1}, x_t]\right)$$

$$r_t = \sigma\left(W_r \cdot [h_{t-1}, x_t]\right)$$

$$\tilde{h}_t = \tanh\left(W \cdot [r_t * h_{t-1}, x_t]\right)$$

$$h_t = (1 - z_t) * h_{t-1} + z_t * \tilde{h}_t$$

References

- http://aikorea.org/blog/rnn-tutorial-1/
- http://www.wildml.com/2015/09/recurrent-neural-networks-tutorial-part-1-introduction-to-rnns/
- http://ir.hit.edu.cn/~jguo/docs/notes/bptt.pdf
- http://minds.jacobs-university.de/sites/default/files/uploads/papers/ESNTutorialRev.pdf
- http://karpathy.github.io/2015/05/21/rnn-effectiveness/
- http://colah.github.io/posts/2015-08-Understanding-LSTMs
- http://deeplearning4j.org/lstm.html
- http://cs231n.stanford.edu/slides/winter1516_lecture10.pdf
- http://peterroelants.github.io/posts/rnn_implementation_part01/
- http://andrew.gibiansky.com/blog/machine-learning/recurrent-neural-networks/

THANK YOU!